1. Lowercasing

Converts all text to lowercase.

Helps treat words like "Hello" and "hello" as the same word.

```
text = "Hello! this is a SAMPLE Email to check AKHIL BOGGITI"
text_lower = text.lower()
print(text_lower)
```

```
hello! this is a sample email to check akhil boggiti
```

2. Removing Special Characters

Removes punctuation, numbers, and unwanted symbols using Regular Expressions (Regex).

Keeps only alphabetic characters.

```
import re
text = "Hello! this is a SAMPLE Email to check AKHIL BOGGITI"
text_clean = re.sub(r'[^a-z ]', '',text)
print(text_clean)
```

```
ello this is a  mail to check
```

3. Stop Word Removal

Removes common words that add little meaning.

Examples: is, am, are, the, a, an, to, of

```
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords', quiet=True)
stop_words = set(stopwords.words('english'))
text = "my name is akhil"
tokens = []
for word in text.split():
    if word not in stop_words:
        tokens.append(word)
print(tokens)
```

```
['name', 'akhil']
```

4. Tokenization

Splits text into smaller units called tokens (words or sentences)

```
import nltk
nltk.download('punkt_tab', quiet=True)
sentence = "i don't love spam emails, do you?"


#Method:1 basic split (misses contraction)

basic = sentence.split()
print("Basic split:", basic)

#Method:2 NLTK's smart Tokeniser

from nltk.tokenize import word_tokenize
smart = word_tokenize(sentence.lower())
print("NLTK Tokenize:", smart)
```

```
Basic split: ['i', "don't", 'love', 'spam', 'emails,', 'do', 'you?']
NLTK Tokenize: ['i', 'do', "n't", 'love', 'spam', 'emails', ',', 'do', 'you', '?']
```

5. Stemming

Reduces words to their root form by cutting endings.

May not produce actual dictionary words.

6. Lemmatization

Converts words to their meaningful dictionary form (lemma).

More accurate than stemming.

```
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
nltk.download('wordnet', quiet=True)
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
words = ['running', 'studies', 'better', 'caring', 'happily']
for word in words:
    stem = stemmer.stem(word)
    lemma = lemmatizer.lemmatize(word, pos='v')
    print(f'word:{word:12} | stem: {stem:10} | lemma: {lemma}')
```

```
word:running      | stem: run        | lemma: run
word:studies      | stem: studi      | lemma: study
word:better       | stem: better     | lemma: better
word:caring       | stem: care       | lemma: care
word:happily      | stem: happili    | lemma: happily
```

7. Complete Text Preprocessing Pipeline

1. Convert to lowercase
2. Remove special characters
3. Tokenize text
4. Remove stop words
5. Lemmatize words
6. Join tokens back into text

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
nltk.download(['stopwords','wordnet','punkt'], quiet=True)
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z ]', '', text)
    tokens = text.split()
    filtered_tokens = []
    for w in tokens:
        if w not in stop_words:
            filtered_tokens.append(w)
    tokens = filtered_tokens
    lemmatized_tokens = []
    for w in tokens:
        lemmatized_tokens.append(
        lemmatizer.lemmatize(w)
        )
        tokens = lemmatized_tokens
    return ' '.join(tokens)
print(preprocess('I am running to the stores to buy some grocery'))
```

```
running store buy grocery
```

```
from sklearn.feature_extraction.text import CountVectorizer
emails = [
    'cricket match cricket match cricket',
    'free win money free win money win',
    'cricket match free win money cricket'
]
cv = CountVectorizer()
X = cv.fit_transform(emails).toarray()
print('Vocabulary:', cv.get_feature_names_out())
```

```
print('BoW matrix:')
print(X)
```

```
Vocabulary: ['cricket' 'free' 'match' 'money' 'win']
BoW matrix:
[[3 0 2 0 0]
 [0 2 0 2 3]
 [2 1 1 1 1]]
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
emails = [
    'cricket match cricket match cricket',
    'free win money free win money win',
    'cricket match free win money cricket'
]
tfidf = TfidfVectorizer(sublinear_tf=True, max_df=0.95, min_df=1)
X_tfidf = tfidf.fit_transform(emails).toarray()
print('Feature names:', tfidf.get_feature_names_out())
print('TF-IDF matrix (rounded):')
import numpy as np
print(np.round(X_tfidf, 3))
```

```
Feature names: ['cricket' 'free' 'match' 'money' 'win']
TF-IDF matrix (rounded):
[[0.778 0.    0.628 0.    0.   ]
 [0.    0.532 0.    0.532 0.659]
 [0.646 0.382 0.382 0.382 0.382]]
```